

基础数据结构

Data Structure into Door

wcr

2021 年 3 月 1 日

- 数据结构是计算机存储、组织数据的方式。数据结构是指相互之间存在一种或多种特定关系的数据元素的集合。通常情况下，精心选择的数据结构可以带来更高的运行或者存储效率。数据结构往往同高效的检索算法和索引技术有关。
- Nicklaus Wirth（1984 年图灵奖获得者）：算法 + 数据结构 = 程序

目录

- 1 栈
 - 经典应用
 - 单调栈
- 2 队列
- 3 链表

- 4 桶
- 5 并查集
- 6 堆
- 7 哈希表

- 栈是一种特殊的线性表，特殊之处在于它只允许在表的一端进行插入和删除运算。这一端被称为栈顶，另一端称为栈底。

- 栈是一种特殊的线性表，特殊之处在于它只允许在表的一端进行插入和删除运算。这一端被称为栈顶，另一端称为栈底。
- 即“先进后出”。

栈

- 栈是一种特殊的线性表，特殊之处在于它只允许在表的一端进行插入和删除运算。这一端被称为栈顶，另一端称为栈底。
- 即“先进后出”。
- 利用栈可以实现深度优先搜索 dfs。

括号匹配

- 判断一个括号序列是否匹配。

括号匹配

- 从左向右扫描表达式。

括号匹配

- 从左向右扫描表达式。
- 如果是左括号，则将其压入栈中；

括号匹配

- 从左向右扫描表达式。
- 如果是左括号，则将其压入栈中；
- 如果是右括号，这个右括号应和栈顶的左括号匹配，弹出栈顶的左括号。

括号匹配

- 从左向右扫描表达式。
- 如果是左括号，则将其压入栈中；
- 如果是右括号，这个右括号应和栈顶的左括号匹配，弹出栈顶的左括号。
- 若此时栈内为空，说明当前右括号失配，返回无解信息。

括号匹配

- 从左向右扫描表达式。
- 如果是左括号，则将其压入栈中；
- 如果是右括号，这个右括号应和栈顶的左括号匹配，弹出栈顶的左括号。
- 若此时栈内为空，说明当前右括号失配，返回无解信息。
- 扫描结束时，若栈非空，说明有左括号失配，返回无解信息。

表达式求值

- 给出一个合法的仅包含数字、四则运算符和小括号的表达式，求出它的值。

表达式求值

- 开始先在表达式前后加上一对括号方便计算

表达式求值

- 开始先在表达式前后加上一对括号方便计算
- 开两个栈，分别是符号栈和数字栈。

表达式求值

- 开始先在表达式前后加上一对括号方便计算
- 开两个栈，分别是符号栈和数字栈。
- 从左向右扫描表达式。

表达式求值

- 开始先在表达式前后加上一对括号方便计算
- 开两个栈，分别是符号栈和数字栈。
- 从左向右扫描表达式。
- 若当前字符为数字，则加入到当前数字末尾，即 $now \leftarrow now \times 10 + a_i$

表达式求值

- 开始先在表达式前后加上一对括号方便计算
- 开两个栈，分别是符号栈和数字栈。
- 从左向右扫描表达式。
- 若当前字符为数字，则加入到当前数字末尾，即 $now \leftarrow now \times 10 + a_i$
- 若当前字符为运算符，将 now 压入数字栈并清零，将符号栈顶的所有优先级大于等于当前运算符的所有元素弹出并运算。再压入当前运算符。

表达式求值

- 开始先在表达式前后加上一对括号方便计算
- 开两个栈，分别是符号栈和数字栈。
- 从左向右扫描表达式。
- 若当前字符为数字，则加入到当前数字末尾，即 $now \leftarrow now \times 10 + a_i$
- 若当前字符为运算符，将 now 压入数字栈并清零，将符号栈顶的所有优先级大于等于当前运算符的所有元素弹出并运算。再压入当前运算符。
- 若当前字符为左括号，直接将其压入符号栈。

表达式求值

- 开始先在表达式前后加上一对括号方便计算
- 开两个栈，分别是符号栈和数字栈。
- 从左向右扫描表达式。
- 若当前字符为数字，则加入到当前数字末尾，即 $now \leftarrow now \times 10 + a_i$
- 若当前字符为运算符，将 now 压入数字栈并清零，将符号栈顶的所有优先级大于等于当前运算符的所有元素弹出并运算。再压入当前运算符。
- 若当前字符为左括号，直接将其压入符号栈。
- 若当前字符为右括号，将 now 压入数字栈并清零，不断弹出两个栈内的元素直到遇到第一个左括号。

表达式求值

- 开始先在表达式前后加上一对括号方便计算
- 开两个栈，分别是符号栈和数字栈。
- 从左向右扫描表达式。
- 若当前字符为数字，则加入到当前数字末尾，即 $now \leftarrow now \times 10 + a_i$
- 若当前字符为运算符，将 now 压入数字栈并清零，将符号栈顶的所有优先级大于等于当前运算符的所有元素弹出并运算。再压入当前运算符。
- 若当前字符为左括号，直接将其压入符号栈。
- 若当前字符为右括号，将 now 压入数字栈并清零，不断弹出两个栈内的元素直到遇到第一个左括号。
- 最后数字栈内应只剩下一个元素，即为表达式的值

单调栈

- 顾名思义，即栈内元素单调的栈。

单调栈

- 顾名思义，即栈内元素单调的栈。
- 最经典的应用就是能对于一个序列中的每个数，求出它左边/右边第一个大于/小于它的数。

单调栈

- 顾名思义，即栈内元素单调的栈。
- 最经典的应用就是能对于一个序列中的每个数，求出它左边/右边第一个大于/小于它的数。
- 如何实现？

- 以求左边第一个比它小的数为例。

单调栈

- 以求左边第一个比它小的数为例。
- 从左到右扫描数组。

单调栈

- 以求左边第一个比它小的数为例。
- 从左到右扫描数组。
- 考虑插入当前的数。如果栈顶元素大于等于当前元素，则栈顶元素显然不可能成为答案了。所以不断弹出栈顶元素，直到栈空或者栈顶元素小于当前元素。

单调栈

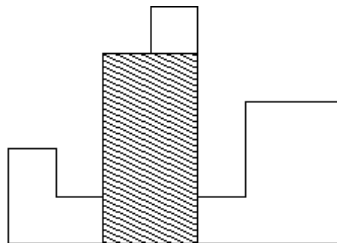
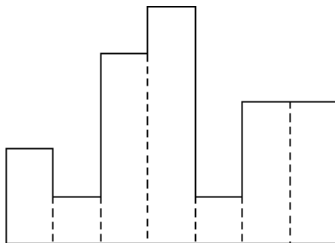
- 以求左边第一个比它小的数为例。
- 从左到右扫描数组。
- 考虑插入当前的数。如果栈顶元素大于等于当前元素，则栈顶元素显然不可能成为答案了。所以不断弹出栈顶元素，直到栈空或者栈顶元素小于当前元素。
- 此时的栈顶元素即为当前左边第一个比当前元素小的数，存储答案并在栈顶加入当前元素即可。

单调栈

- 以求左边第一个比它小的数为例。
- 从左到右扫描数组。
- 考虑插入当前的数。如果栈顶元素大于等于当前元素，则栈顶元素显然不可能成为答案了。所以不断弹出栈顶元素，直到栈空或者栈顶元素小于当前元素。
- 此时的栈顶元素即为当前左边第一个比当前元素小的数，存储答案并在栈顶加入当前元素即可。
- 时间复杂度 $O(n)$

poj2559 Largest Rectangle in a Histogram

- 给出一个柱状图，求其中最大的矩形的面积。
- 柱子个数 $\leq 10^5$



poj2559 Largest Rectangle in a Histogram

- 考虑一个最大的矩形，显然上下左右都不能再扩展。所以上边界一定会顶住某个柱子的上界。

poj2559 Largest Rectangle in a Histogram

- 考虑一个最大的矩形，显然上下左右都不能再扩展。所以上边界一定会顶住某个柱子的上界。
- 所以可以枚举矩形顶住了哪个柱子的上界，显然左右最多只能拓展到当前柱子左边右边的第一个比它低的柱子之前。计算得矩形面积更新答案即可。

poj2559 Largest Rectangle in a Histogram

- 考虑一个最大的矩形，显然上下左右都不能再扩展。所以上边界一定会顶住某个柱子的上界。
- 所以可以枚举矩形顶住了哪个柱子的上界，显然左右最多只能拓展到当前柱子左边右边的第一个比它低的柱子之前。计算得矩形面积更新答案即可。
- 至于左右第一个更低的柱子可以先用单调栈预处理得到。

- CSP-S2019 括号树
- NOIP2005 等价表达式
- USACO08JAN Artificial Lake G
- ZJOI2007 棋盘制作

目录

1 栈

2 队列

- 循环队列
- 双端队列
- 单调队列

3 链表

4 桶

5 并查集

6 堆

7 哈希表

- 队列是一种特殊的线性表，特殊之处在于它只允许在表的前端进行删除操作，而在表的后端进行插入操作。

队列

- 队列是一种特殊的线性表，特殊之处在于它只允许在表的前端进行删除操作，而在表的后端进行插入操作。
- 即“先进先出”。

队列

- 队列是一种特殊的线性表，特殊之处在于它只允许在表的前端进行删除操作，而在表的后端进行插入操作。
- 即“先进先出”。
- 利用队列可以实现广度优先搜索 bfs。

- 给出由 n 个非负整数组成的数集。每单位时间下列事件按顺序发生：
 - 将最大的数取出，设为 m ;
 - 数集中所有数字增加 q ;
 - 将 $\lfloor \frac{um}{v} \rfloor$ 和 $m - \lfloor \frac{um}{v} \rfloor$ 加入数集。
- 请求出每单位时间的 m 和 m 时刻后数集中的所有数。
- 其中 q, u, v 均为给定非负常数，且 $u < v$ 。 $n \leq 10^5, m \leq 7 \times 10^6$ 。有某种控制输出规模的机制。

- 直接用堆维护是 $\mathcal{O}(n \log n)$ 的，显然会 TLE。考虑线性做法。

- 直接用堆维护是 $\mathcal{O}(n \log n)$ 的，显然会 TLE。考虑线性做法。
- 观察到当前新产生的第一、二种数一定比上一次产生的同种小。

- 直接用堆维护是 $\mathcal{O}(n \log n)$ 的，显然会 TLE。考虑线性做法。
- 观察到当前新产生的第一、二种数一定比上一次产生的同种小。
- 因此可以开三个队列，分别维护初始数集、产生的第一种数、产生的第二种数排序后的结果。

- 直接用堆维护是 $\mathcal{O}(n \log n)$ 的，显然会 TLE。考虑线性做法。
- 观察到当前新产生的第一、二种数一定比上一次产生的同种小。
- 因此可以开三个队列，分别维护初始数集、产生的第一种数、产生的第二种数排序后的结果。
- 每次比较三个队首得到 m ，弹出并将新产生的两个数丢到两个队列的队尾。

- 关于性质的证明：（只讨论第一种、第二种同理）

- 关于性质的证明：（只讨论第一种、第二种同理）
- 设上次选的数为 m_1 ，当前选的是 m_2 。显然有 $q + m_1 \geq m_2$ 。

- 关于性质的证明：（只讨论第一种、第二种同理）
- 设上次选的数为 m_1 ，当前选的是 m_2 。显然有 $q + m_1 \geq m_2$ 。
- 则上次产生的数为 $m_1p + q$ ，当前产生的是 m_2p 。（因为 $\lfloor \frac{um}{v} \rfloor$ 相当于 m 乘上一个分数，所以写为 mp ）

- 关于性质的证明：（只讨论第一种、第二种同理）
- 设上次选的数为 m_1 ，当前选的是 m_2 。显然有 $q + m_1 \geq m_2$ 。
- 则上次产生的数为 $m_1p + q$ ，当前产生的是 m_2p 。（因为 $\lfloor \frac{um}{v} \rfloor$ 相当于 m 乘上一个分数，所以写为 mp ）
- 于是有 $m_2p \leq (m_1 + q)p = m_1p + qp$

- 关于性质的证明：（只讨论第一种、第二种同理）
- 设上次选的数为 m_1 ，当前选的是 m_2 。显然有 $q + m_1 \geq m_2$ 。
- 则上次产生的数为 $m_1p + q$ ，当前产生的是 m_2p 。（因为 $\lfloor \frac{um}{v} \rfloor$ 相当于 m 乘上一个分数，所以写为 mp ）
- 于是有 $m_2p \leq (m_1 + q)p = m_1p + qp$
- 因为 $p < 1$ ，所以有 $m_1p + q \geq m_1p + qp \geq m_2p$

循环队列

- 一个优化空间的小技巧。

循环队列

- 一个优化空间的小技巧。
- 使用数组模拟队列时我们会发现整个队列整体向数组的尾部移动，而数组前端有大量的空间空闲。

循环队列

- 一个优化空间的小技巧。
- 使用数组模拟队列时我们会发现整个队列整体向数组的尾部移动，而数组前端有大量的空间空闲。
- 所以当首尾指针越界时，令其移动到数组的头部，重新利用数组前端的空间。

循环队列

- 一个优化空间的小技巧。
- 使用数组模拟队列时我们会发现整个队列整体向数组的尾部移动，而数组前端有大量的空间空闲。
- 所以当首尾指针越界时，令其移动到数组的头部，重新利用数组前端的空间。
- 这样就相当于将数组尾部的下一个位置看作了数组的头部，故此得名“循环队列”。

双端队列

- 顾名思义，即头尾都能进行插入删除的队列。

双端队列

- 顾名思义，即头尾都能进行插入删除的队列。
- 有啥用？

- 给出一张 n 个点 m 条边的边权只有 0/1 的有向图，求点 1 到点 n 的最短路。
- $n \leq 5 \times 10^6, m \leq 10^7$
- 常数/读入效率等因素可以不用考虑，你只需要保证时间复杂度正确即可。

Here is the Solution.

- 考虑双向 bfs。开始双端队列内只有一个点 1。

Here is the Solution.

- 考虑双向 bfs。开始双端队列内只有一个点 1。
- 每次从队首取出一个元素，以它为起点进行松弛。

Here is the Solution.

- 考虑双向 bfs。开始双端队列内只有一个点 1。
- 每次从队首取出一个元素，以它为起点进行松弛。
- 若当前边边权为 0，且连向的点 v 的最短路被更新，则将 v 插到队首。

Here is the Solution.

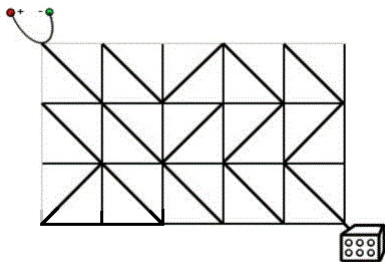
- 考虑双向 bfs。开始双端队列内只有一个点 1。
- 每次从队首取出一个元素，以它为起点进行松弛。
- 若当前边边权为 0，且连向的点 v 的最短路被更新，则将 v 插到队首。
- 若当前边边权为 1，且连向的点 v 的最短路被更新，则将 v 插到队尾。

Here is the Solution.

- 考虑双向 bfs。开始双端队列内只有一个点 1。
- 每次从队首取出一个元素，以它为起点进行松弛。
- 若当前边边权为 0，且连向的点 v 的最短路被更新，则将 v 插到队首。
- 若当前边边权为 1，且连向的点 v 的最短路被更新，则将 v 插到队尾。
- 正确性显然。

BalticOI 2011 Day1 Switch the Lamp On

- 有 $n \times m$ 个正方形电路元件，排列成 n 行 m 列。
- 每个电路元件连通了它的一条对角线。
- 每次操作你可以使一个电路元件旋转 90° 。
- 位于 $(1, 1)$ 的元件的左上角连接了电源，位于 (n, m) 的元件的右下角连接了灯泡。
- 求最少操作多少次使得电源与灯连通。



- 建 $(n + 1) \times (m + 1)$ 个点，表示网格图的每个格点。

- 建 $(n + 1) \times (m + 1)$ 个点，表示网格图的每个格点。
- 对于每个元件的每条对角线，若已经连通则连边权为 0 的边，否则连边权为 1 的边，表示需要操作一次。

- 建 $(n + 1) \times (m + 1)$ 个点，表示网格图的每个格点。
- 对于每个元件的每条对角线，若已经连通则连边权为 0 的边，否则连边权为 1 的边，表示需要操作一次。
- 然后直接跑双向 bfs 求出 $(1, 1)$ 到 $(n + 1, m + 1)$ 的最短路即可

单调队列

- 顾名思义，即队内元素单调的队列。

单调队列

- 顾名思义，即队内元素单调的队列。
- 要注意的是它本质上其实是双端队列。

单调队列

- 顾名思义，即队内元素单调的队列。
- 要注意的是它本质上其实是双端队列。
- 可以解决一类 `rmq` 问题。

单调队列

- 顾名思义，即队内元素单调的队列。
- 要注意的是它本质上其实是双端队列。
- 可以解决一类 `rmq` 问题。
- 具体是啥？先来道例题。

- 给出一个长度为 n 的序列 a 和整数 k 。
- 定义 $b_i = \min_{j=i}^{i+k-1} a_j$ 。
- 定义 $c_i = \max_{j=i}^{i+k-1} a_j$ 。
- 求 $b_{1..n-k+1}, c_{1..n-k+1}$ 。

- 考虑维护一个队内元素递增的队列。队内元素即为当前区间最优元素的候选队列。

- 考虑维护一个队内元素递增的队列。队内元素即为当前区间最优元素的候选队列。
- 以求 b 序列为例。从左到右扫描数组。

- 考虑维护一个队内元素递增的队列。队内元素即为当前区间最优元素的候选队列。
- 以求 b 序列为例。从左到右扫描数组。
- 如果队尾元素大于等于当前元素，那么队尾元素显然不可能成为最小值了，那么不断弹出队尾元素直到队空或队尾元素小于当前元素。

- 考虑维护一个队内元素递增的队列。队内元素即为当前区间最优元素的候选队列。
- 以求 b 序列为例。从左到右扫描数组。
- 如果队尾元素大于等于当前元素，那么队尾元素显然不可能成为最小值了，那么不断弹出队尾元素直到队空或队尾元素小于当前元素。
- 此时向队尾插入当前元素。

- 考虑维护一个队内元素递增的队列。队内元素即为当前区间最优元素的候选队列。
- 以求 b 序列为例。从左到右扫描数组。
- 如果队尾元素大于等于当前元素，那么队尾元素显然不可能成为最小值了，那么不断弹出队尾元素直到队空或队尾元素小于当前元素。
- 此时向队尾插入当前元素。
- 可以发现队列内的元素是单调递增的。

- 考虑维护一个队内元素递增的队列。队内元素即为当前区间最优元素的候选队列。
- 以求 b 序列为例。从左到右扫描数组。
- 如果队尾元素大于等于当前元素，那么队尾元素显然不可能成为最小值了，那么不断弹出队尾元素直到队空或队尾元素小于当前元素。
- 此时向队尾插入当前元素。
- 可以发现队列内的元素是单调递增的。
- 队首元素显然就是当前区间的最小值。

- 考虑维护一个队内元素递增的队列。队内元素即为当前区间最优元素的候选队列。
- 以求 b 序列为例。从左到右扫描数组。
- 如果队尾元素大于等于当前元素，那么队尾元素显然不可能成为最小值了，那么不断弹出队尾元素直到队空或队尾元素小于当前元素。
- 此时向队尾插入当前元素。
- 可以发现队列内的元素是单调递增的。
- 队首元素显然就是当前区间的最小值。
- 但此时队首元素可能已经不在当前区间中了，因此需要判断队首元素若不在询问区间内则不断弹出队首直到合法。

- 考虑维护一个队内元素递增的队列。队内元素即为当前区间最优元素的候选队列。
- 以求 b 序列为例。从左到右扫描数组。
- 如果队尾元素大于等于当前元素，那么队尾元素显然不可能成为最小值了，那么不断弹出队尾元素直到队空或队尾元素小于当前元素。
- 此时向队尾插入当前元素。
- 可以发现队列内的元素是单调递增的。
- 队首元素显然就是当前区间的最小值。
- 但此时队首元素可能已经不在当前区间中了，因此需要判断队首元素若不在询问区间内则不断弹出队首直到合法。
- 可以发现使用单调队列的条件是询问区间的左右端点都单调不降。

- 给出一个长度为 n 的序列 a ，你需要选择 m 个元素，使得每连续的 k 个元素中至少有一个数被选中。
- 求选中的所有数的和的最大值。
- $1 \leq k, m \leq n \leq 5000, 1 \leq a_i \leq 10^9$

- 单调队列优化 dp 板子题。

- 单调队列优化 dp 板子题。
- 设 $f_{i,j}$ 表示前 i 个数里选了 j 个数，强制第 i 个数选的最大和。

- 单调队列优化 dp 板子题。
- 设 $f_{i,j}$ 表示前 i 个数里选了 j 个数，强制第 i 个数选的最大和。
- 显然有 $f_{i,j} = \max_{t=i-k}^{i-1} f_{t,j-1} + a_i$ 。

- 单调队列优化 dp 板子题。
- 设 $f_{i,j}$ 表示前 i 个数里选了 j 个数，强制第 i 个数选的最大和。
- 显然有 $f_{i,j} = \max_{t=i-k}^{i-1} f_{t,j-1} + a_i$ 。
- 先枚举 j 再枚举 i ，然后 $\max$ 里的东西就可以单调队列优化转移了。

- 有 n 棵树，第 i 棵树的高度是 d_i 。
- 马阿克开始在第一棵树，她要去第 n 棵树。
- 如果马阿克在第 i 棵树，那么她可以跳到第 $i+1, i+2, \dots, i+k$ 棵树。
- 如果她跳到一棵不矮于当前树的树，那么她的劳累值会增加 1；否则不会。
- 求马阿克到达第 n 棵树的最小劳累值。
- q 组询问，每组给出不同的 k
- $n \leq 10^6, q \leq 25$ 。

- 设 f_i 表示走到第 i 个点最小的疲劳值。

- 设 f_i 表示走到第 i 个点最小的疲劳值。
- 考虑怎样的转移点是最优的。可以发现是优先疲劳值最小，在疲劳值相同时保证高度最大。

- 设 f_i 表示走到第 i 个点最小的疲劳值。
- 考虑怎样的转移点是最优的。可以发现是优先疲劳值最小，在疲劳值相同时保证高度最大。
- 原因是两个疲劳值不同的显然是疲劳值较小的更优，因为疲劳值最多只会加 1。

- 设 f_i 表示走到第 i 个点最小的疲劳值。
- 考虑怎样的转移点是最优的。可以发现是优先疲劳值最小，在疲劳值相同时保证高度最大。
- 原因是两个疲劳值不同的显然是疲劳值较小的更优，因为疲劳值最多只会加 1。
- 而在疲劳值相同时显然是树高更高的更优。

- 设 f_i 表示走到第 i 个点最小的疲劳值。
- 考虑怎样的转移点是最优的。可以发现是优先疲劳值最小，在疲劳值相同时保证高度最大。
- 原因是两个疲劳值不同的显然是疲劳值较小的更优，因为疲劳值最多只会加 1。
- 而在疲劳值相同时显然是树高更高的更优。
- 按照这个策略用单调队列维护即可。

- HAOI2007 理想的正方形
- POI2011 TEM-Temperature
- CSP-S2019 划分

目录

1 栈

2 队列

3 链表

- 块状链表

4 桶

5 并查集

6 堆

7 哈希表

- 链表是一种物理存储单元上非连续、非顺序的存储结构，数据元素的逻辑顺序是通过链表中的指针链接次序实现的。

链表

- 链表是一种物理存储单元上非连续、非顺序的存储结构，数据元素的逻辑顺序是通过链表中的指针链接次序实现的。
- 链表由一系列结点组成，每个结点包括两个部分：一个是存储数据元素的数据域，另一个是存储相邻结点地址的指针域。

链表

- 链表是一种物理存储单元上非连续、非顺序的存储结构，数据元素的逻辑顺序是通过链表中的指针链接次序实现的。
- 链表由一系列结点组成，每个结点包括两个部分：一个是存储数据元素的数据域，另一个是存储相邻结点地址的指针域。
- 分为单向链表和双向链表。

链表

- 链表是一种物理存储单元上非连续、非顺序的存储结构，数据元素的逻辑顺序是通过链表中的指针链接次序实现的。
- 链表由一系列结点组成，每个结点包括两个部分：一个是存储数据元素的数据域，另一个是存储相邻结点地址的指针域。
- 分为单向链表和双向链表。
- 经典的应用：链式前向星存图。

- 给出一个 $n \times m$ 的矩阵和 Q 次操作，每次操作交换两个不重合且不相邻的两个完全相同的矩形中的所有元素，求最后的结果。
- $n, m \leq 1000, Q \leq 10000$ 。

- 二维链表即可，时间复杂度 $O((n + m)Q)$

块状链表

- 众所周知

块状链表

- 众所周知
- 链表插入删除是 $\mathcal{O}(1)$ 的，但随机访问是 $\mathcal{O}(n)$ 的。

块状链表

- 众所周知
- 链表插入删除是 $\mathcal{O}(1)$ 的，但随机访问是 $\mathcal{O}(n)$ 的。
- 数组随机访问是 $\mathcal{O}(1)$ 的，但插入删除是 $\mathcal{O}(n)$ 的。

块状链表

- 众所周知
- 链表插入删除是 $\mathcal{O}(1)$ 的，但随机访问是 $\mathcal{O}(n)$ 的。
- 数组随机访问是 $\mathcal{O}(1)$ 的，但插入删除是 $\mathcal{O}(n)$ 的。
- 而块状链表是把这两个数据结构结合一下，两个操作都是 $\mathcal{O}(\sqrt{n})$ 。

块状链表

- 考虑设置一个阈值 B ，维护一个每个元素都是一个长度接近 B 的数组的链表。

块状链表

- 考虑设置一个阈值 B ，维护一个每个元素都是一个长度接近 B 的数组的链表。
- 插入删除时遍历链表找到位置并暴力修改，复杂度 $\mathcal{O}(\max(B, \frac{n}{B}))$ 。

块状链表

- 考虑设置一个阈值 B ，维护一个每个元素都是一个长度接近 B 的数组的链表。
- 插入删除时遍历链表找到位置并暴力修改，复杂度 $\mathcal{O}(\max(B, \frac{n}{B}))$ 。
- 但是修改完我们需要保证每个结点内数组的长度接近 B ，所以可以当 B 长度超过 $2B$ 时分裂当前结点。

块状链表

- 考虑设置一个阈值 B ，维护一个每个元素都是一个长度接近 B 的数组的链表。
- 插入删除时遍历链表找到位置并暴力修改，复杂度 $\mathcal{O}(\max(B, \frac{n}{B}))$ 。
- 但是修改完我们需要保证每个结点内数组的长度接近 B ，所以可以当 B 长度超过 $2B$ 时分裂当前结点。
- 最多分裂 n/B 次，总时间复杂度 $\mathcal{O}(\frac{n}{B} \times B)$ 。

块状链表

- 考虑设置一个阈值 B ，维护一个每个元素都是一个长度接近 B 的数组的链表。
- 插入删除时遍历链表找到位置并暴力修改，复杂度 $\mathcal{O}(\max(B, \frac{n}{B}))$ 。
- 但是修改完我们需要保证每个结点内数组的长度接近 B ，所以可以当 B 长度超过 $2B$ 时分裂当前结点。
- 最多分裂 n/B 次，总时间复杂度 $\mathcal{O}(\frac{n}{B} \times B)$ 。
- 显然 B 取 $\sqrt{n}$ 时最优。

块状链表

- 考虑设置一个阈值 B ，维护一个每个元素都是一个长度接近 B 的数组的链表。
- 插入删除时遍历链表找到位置并暴力修改，复杂度 $\mathcal{O}(\max(B, \frac{n}{B}))$ 。
- 但是修改完我们需要保证每个结点内数组的长度接近 B ，所以可以当 B 长度超过 $2B$ 时分裂当前结点。
- 最多分裂 n/B 次，总时间复杂度 $\mathcal{O}(\frac{n}{B} \times B)$ 。
- 显然 B 取 $\sqrt{n}$ 时最优。
- 模板题：poj2887 Big String

目录

- 1 栈
- 2 队列
- 3 链表

- 4 桶
- 5 并查集
- 6 堆
- 7 哈希表

- 根据数值直接进行访问的数据结构。

- 根据数值直接进行访问的数据结构。
- 经典的应用：桶排序、基数排序。

- 给出一个长度为 n 的数列，数列中的数字不超过 m 。要求在原序列中取出一个尽量长的子序列，使子序列中相邻两个数的最大公约数全部大于 L 。
- $n \leq 50000, m \leq 1000000$ 。

- 设 f_i 表示强制选第 i 个数，前 i 个数中最多能选出多少数。

- 设 f_i 表示强制选第 i 个数，前 i 个数中最多能选出多少数。
- 维护一个桶，第 i 个位置表示包含 i 这个因子的数最大的 f 值是多少。

- 设 f_i 表示强制选第 i 个数，前 i 个数中最多能选出多少数。
- 维护一个桶，第 i 个位置表示包含 i 这个因子的数最大的 f 值是多少。
- 转移时枚举因子，从桶里找最大值即可。

- 设 f_i 表示强制选第 i 个数，前 i 个数中最多能选出多少数。
- 维护一个桶，第 i 个位置表示包含 i 这个因子的数最大的 f 值是多少。
- 转移时枚举因子，从桶里找最大值即可。
- 转移完再将当前数加入桶中，同样枚举因子即可。

目录

- 1 栈
- 2 队列
- 3 链表
- 4 桶

- 5 并查集
 - 种类并查集
 - 带权并查集

- 6 堆

- 7 哈希表

- 一种树形数据结构，用于维护一些不交集的合并及查询的问题。

并查集

- 一种树形数据结构，用于维护一些不交集的合并及查询的问题。
- 板子过于基础，相信大家都会，就不讲了。

并查集

- 一种树形数据结构，用于维护一些不交集的合并及查询的问题。
- 板子过于基础，相信大家都会，就不讲了。
- 有两种优化方法：路径压缩和按秩合并。

并查集

- 一种树形数据结构，用于维护一些不交集的合并及查询的问题。
- 板子过于基础，相信大家都会，就不讲了。
- 有两种优化方法：路径压缩和按秩合并。
- 两个分开来用都是 $\mathcal{O}(\log)$ 的，一起用是 $\mathcal{O}(\alpha)$ 的。

并查集

- 一种树形数据结构，用于维护一些不交集的合并及查询的问题。
- 板子过于基础，相信大家都会，就不讲了。
- 有两种优化方法：路径压缩和按秩合并。
- 两个分开来用都是 $\mathcal{O}(\log)$ 的，一起用是 $\mathcal{O}(\alpha)$ 的。
- 证明讲课人不会。

并查集

- 一种树形数据结构，用于维护一些不交集的合并及查询的问题。
- 板子过于基础，相信大家都会，就不讲了。
- 有两种优化方法：路径压缩和按秩合并。
- 两个分开来用都是 $\mathcal{O}(\log)$ 的，一起用是 $\mathcal{O}(\alpha)$ 的。
- 证明讲课人不会。
- 在一些情况下加路径压缩会出事，需要写按秩合并。

- 给出一张 n 个点 m 条边的无向图，给出 k 次删点操作，求每次操作后以及原图中的连通块个数。
- $k \leq m \leq 2 \times 10^5, n \leq 2m$

- 删点显然非常难搞，所以考虑将所有操作离线下来然后倒过来做。

- 删点显然非常难搞，所以考虑将所有操作离线下来然后倒过来做。
- 然后操作就变成了加点，直接加边加边加边，并查集查询。

- 给定 n 个人，他们之间有两种关系，朋友与敌对。可以肯定的是：
 - 与我的朋友是朋友的人是我的朋友。
 - 与我敌对的人有敌对关系的人是我的朋友。
- 现在这 n 个人进行组团，两个人在一个团队内当且仅当他们是朋友。
- 给出 m 对关系，求最多的团体数。
- $n, m \leq 10^5$ 。

- 种类并查集模板。给每个人建两个点，分别为 i 和 $i + n$ 。

- 种类并查集模板。给每个人建两个点，分别为 i 和 $i + n$ 。
- 处理时：

- 种类并查集模板。给每个人建两个点，分别为 i 和 $i + n$ 。
- 处理时：
 - 如果两个人 i, j 是朋友，合并 i 和 j ， $i + n$ 和 $j + n$ 。

- 种类并查集模板。给每个人建两个点，分别为 i 和 $i + n$ 。
- 处理时：
 - 如果两个人 i, j 是朋友，合并 i 和 j ， $i + n$ 和 $j + n$ 。
 - 如果两个人 i, j 是敌人，合并 i 和 $j + n$ ， $i + n$ 和 j 。

- 种类并查集模板。给每个人建两个点，分别为 i 和 $i + n$ 。
- 处理时：
 - 如果两个人 i, j 是朋友，合并 i 和 j ， $i + n$ 和 $j + n$ 。
 - 如果两个人 i, j 是敌人，合并 i 和 $j + n$ ， $i + n$ 和 j 。
- 查询时：

- 种类并查集模板。给每个人建两个点，分别为 i 和 $i + n$ 。
- 处理时：
 - 如果两个人 i, j 是朋友，合并 i 和 j ， $i + n$ 和 $j + n$ 。
 - 如果两个人 i, j 是敌人，合并 i 和 $j + n$ ， $i + n$ 和 j 。
- 查询时：
 - 如果两个人 i, j 是朋友，那么 i 和 j 应该连通。

- 种类并查集模板。给每个人建两个点，分别为 i 和 $i + n$ 。
- 处理时：
 - 如果两个人 i, j 是朋友，合并 i 和 j ， $i + n$ 和 $j + n$ 。
 - 如果两个人 i, j 是敌人，合并 i 和 $j + n$ ， $i + n$ 和 j 。
- 查询时：
 - 如果两个人 i, j 是朋友，那么 i 和 j 应该连通。
 - 如果两个人 i, j 是敌人，那么 i 和 $j + n$ 应该连通。

- 种类并查集模板。给每个人建两个点，分别为 i 和 $i + n$ 。
- 处理时：
 - 如果两个人 i, j 是朋友，合并 i 和 j ， $i + n$ 和 $j + n$ 。
 - 如果两个人 i, j 是敌人，合并 i 和 $j + n$ ， $i + n$ 和 j 。
- 查询时：
 - 如果两个人 i, j 是朋友，那么 i 和 j 应该连通。
 - 如果两个人 i, j 是敌人，那么 i 和 $j + n$ 应该连通。
- 正确性显然。

- 有三类动物 A, B, C , 共有 n 只, 给出两种关系:
 - 1 $x y$: $x y$ 同类。
 - 2 $x y$: x 吃 y 。
- 对于假话的定义:
 - 当前的话与前面的某些真的话冲突, 就是假话;
 - 当前的话中 x 或 y 比 n 大, 就是假话;
 - 当前的话表示 x 吃 x , 就是假话。
- 现在给出 k 句这样的关系, 求假话个数。
- $n \leq 5 \times 10^4, k \leq 10^5$

- 和上一题相似的思路。

- 和上一题相似的思路。
- 考虑每个动物开三个点分别表示它是 A, B, C 三类动物的情况。

- 和上一题相似的思路。
- 考虑每个动物开三个点分别表示它是 A, B, C 三类动物的情况。
- 处理关系时：

- 和上一题相似的思路。
- 考虑每个动物开三个点分别表示它是 A, B, C 三类动物的情况。
- 处理关系时：
 - 如果 i, j 是同类，合并 i 和 j ， $i + n$ 和 $j + n$ ， $i + 2n$ 和 $j + 2n$ 。

- 和上一题相似的思路。
- 考虑每个动物开三个点分别表示它是 A, B, C 三类动物的情况。
- 处理关系时：
 - 如果 i, j 是同类，合并 i 和 j , $i + n$ 和 $j + n$, $i + 2n$ 和 $j + 2n$ 。
 - 如果 i 吃 j , 合并 i 和 $j + n$, $i + n$ 和 $j + 2n$, $i + 2n$ 和 j 。

- 和上一题相似的思路。
- 考虑每个动物开三个点分别表示它是 A, B, C 三类动物的情况。
- 处理关系时：
 - 如果 i, j 是同类，合并 i 和 j , $i + n$ 和 $j + n$, $i + 2n$ 和 $j + 2n$ 。
 - 如果 i 吃 j , 合并 i 和 $j + n$, $i + n$ 和 $j + 2n$, $i + 2n$ 和 j 。
- 当然你得先判这句话的真假：

- 和上一题相似的思路。
- 考虑每个动物开三个点分别表示它是 A, B, C 三类动物的情况。
- 处理关系时：
 - 如果 i, j 是同类，合并 i 和 j , $i + n$ 和 $j + n$, $i + 2n$ 和 $j + 2n$ 。
 - 如果 i 吃 j ，合并 i 和 $j + n$, $i + n$ 和 $j + 2n$, $i + 2n$ 和 j 。
- 当然你得先判这句话的真假：
 - i, j 是同类：判断 i 和 $j + n$, i 和 $j + 2n$ 的连通性，若有连通则假。

NOI2001 食物链

- 和上一题相似的思路。
- 考虑每个动物开三个点分别表示它是 A, B, C 三类动物的情况。
- 处理关系时：
 - 如果 i, j 是同类，合并 i 和 j , $i + n$ 和 $j + n$, $i + 2n$ 和 $j + 2n$ 。
 - 如果 i 吃 j ，合并 i 和 $j + n$, $i + n$ 和 $j + 2n$, $i + 2n$ 和 j 。
- 当然你得先判这句话的真假：
 - i, j 是同类：判断 i 和 $j + n$, i 和 $j + 2n$ 的连通性，若有连通则假。
 - i 吃 j ：判断 i 和 j , i 和 $j + 2n$ 的连通性，若有连通则假。

- 和上一题相似的思路。
- 考虑每个动物开三个点分别表示它是 A, B, C 三类动物的情况。
- 处理关系时：
 - 如果 i, j 是同类，合并 i 和 j , $i + n$ 和 $j + n$, $i + 2n$ 和 $j + 2n$ 。
 - 如果 i 吃 j ，合并 i 和 $j + n$, $i + n$ 和 $j + 2n$, $i + 2n$ 和 j 。
- 当然你得先判这句话的真假：
 - i, j 是同类：判断 i 和 $j + n$, i 和 $j + 2n$ 的连通性，若有连通则假。
 - i 吃 j ：判断 i 和 j , i 和 $j + 2n$ 的连通性，若有连通则假。
- 正确性依然很显然。

NOI2002 银河英雄传说

- 开始有 30000 个 zjc 排成 30000 列。
- T 次操作，每个操作有两种格式：
 - $M\ i\ j$: 表示将第 i 个 zjc 所在的列作为一个整体，接到第 j 个 zjc 所在的列之后。
 - $C\ i\ j$: 询问第 i 和 j 个 zjc 是否在同一列中，若在同一列请求出他们之间有多少个 zjc。
- $T \leq 5 \times 10^5$ 。

- 带权并查集板子题。

NOI2002 银河英雄传说

- 带权并查集板子题。
- 主要的难度主要在 C 操作的第二问

NOI2002 银河英雄传说

- 带权并查集板子题。
- 主要的难度主要在 C 操作的第二问
- 首先 i 和 j 之间 zjc 的数量显然可以拆成 1 到 i 和 1 到 j 的个数差 +1。

NOI2002 银河英雄传说

- 带权并查集板子题。
- 主要的难度主要在 C 操作的第二问
- 首先 i 和 j 之间 zjc 的数量显然可以拆成 1 到 i 和 1 到 j 的个数差 +1。
- 在朴素的并查集的基础上，再记录每个点到它父亲的之间有多少个数作为边权。

NOI2002 银河英雄传说

- 带权并查集板子题。
- 主要的难度主要在 C 操作的第二问
- 首先 i 和 j 之间 zjc 的数量显然可以拆成 1 到 i 和 1 到 j 的个数差 +1。
- 在朴素的并查集的基础上，再记录每个点到它父亲的之间有多少个数作为边权。
- 这样只要查询到根结点的路径上的边权和即可。

NOI2002 银河英雄传说

- 带权并查集板子题。
- 主要的难度主要在 C 操作的第二问
- 首先 i 和 j 之间 zjc 的数量显然可以拆成 1 到 i 和 1 到 j 的个数差 +1。
- 在朴素的并查集的基础上，再记录每个点到它父亲的之间有多少个数作为边权。
- 这样只要查询到根结点的路径上的边权和即可。
- 注意路径压缩时需要同时处理一下权值的变化。

- 考虑一个不带路径压缩的并查集，每次操作只会修改一个结点的 father。

可持久化并查集

- 考虑一个不带路径压缩的并查集，每次操作只会修改一个结点的 father。
- 将 father 数组和 size 数组用主席树维护即可。

可持久化并查集

- 考虑一个不带路径压缩的并查集，每次操作只会修改一个结点的 `father`。
- 将 `father` 数组和 `size` 数组用主席树维护即可。
- 不是联赛内容所以不要求掌握。

- UVA11987 Almost Union-Find
- APIO2011 方格染色
- hdu3038 How Many Answers Are Wrong

目录

1 栈

2 队列

3 链表

4 桶

5 并查集

6 堆

- 堆的分类
- 二叉堆
- 对顶堆
- 左偏树

7 哈希表

- 堆是一棵树，其每个结点都有一个键值，且每个结点的键值都大于等于/小于等于其父亲的键值。(这也是堆的性质，以下堆的性质都指这一条)

堆

- 堆是一棵树，其每个结点都有一个键值，且每个结点的键值都大于等于/小于等于其父亲的键值。(这也是堆的性质，以下堆的性质都指这一条)
- 每个结点的键值都大于等于其父亲键值的堆叫做小根堆，否则叫做大根堆。

堆

- 堆是一棵树，其每个结点都有一个键值，且每个结点的键值都大于等于/小于等于其父亲的键值。(这也是堆的性质，以下堆的性质都指这一条)
- 每个结点的键值都大于等于其父亲键值的堆叫做小根堆，否则叫做大根堆。
- 堆主要支持的操作有：插入一个数、查询最小值、删除最小值、合并两个堆、减小一个元素的值。

- 堆是一棵树，其每个结点都有一个键值，且每个结点的键值都大于等于/小于等于其父亲的键值。(这也是堆的性质，以下堆的性质都指这一条)
- 每个结点的键值都大于等于其父亲键值的堆叫做小根堆，否则叫做大根堆。
- 堆主要支持的操作有：插入一个数、查询最小值、删除最小值、合并两个堆、减小一个元素的值。
- 因为大根堆和小根堆的维护原理是相同的，所以本文提到的所有堆都指小根堆。

- 堆是一棵树，其每个结点都有一个键值，且每个结点的键值都大于等于/小于等于其父亲的键值。(这也是堆的性质，以下堆的性质都指这一条)
- 每个结点的键值都大于等于其父亲键值的堆叫做小根堆，否则叫做大根堆。
- 堆主要支持的操作有：插入一个数、查询最小值、删除最小值、合并两个堆、减小一个元素的值。
- 因为大根堆和小根堆的维护原理是相同的，所以本文提到的所有堆都指小根堆。
- 另外习惯上，不加限定提到“堆”时往往都指二叉堆。

堆的分类

操作\数据结构	配对堆	二叉堆	左偏树	二项堆	斐波那契堆
插入 (insert)	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	$O(1)$
查询最小值 (find-min)	$O(1)$	$O(1)$	$O(1)$	$O(\log n)$	$O(1)$
删除最小值 (delete-min)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
合并 (merge)	$O(1)$	$O(n)$	$O(\log n)$	$O(\log n)$	$O(1)$
减小一个元素的值 (decrease-key)	$o(\log n)$ (下界 $\Omega(\log \log n)$, 上界 $O(2^{2\sqrt{\log \log n}})$)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(1)$
是否支持可持久化	×	✓	✓	✓	×

二叉堆

- 二叉堆的结构是一棵二叉树，并且是完全二叉树，每个结点中存有一个元素。

二叉堆

- 二叉堆的结构是一棵二叉树，并且是完全二叉树，每个结点中存有一个元素。
- 它满足堆的性质：父亲的权值不大于儿子的权值。

二叉堆

- 二叉堆的结构是一棵二叉树，并且是完全二叉树，每个结点中存有一个元素。
- 它满足堆的性质：父亲的权值不大于儿子的权值。
- 实际应用的时候可以直接使用 STL 中的优先队列 (priority_queue)

二叉堆的插入操作

- 在最下一层最右边的叶子右方插入当前元素。如果最下一层已满，就新增一层。

二叉堆的插入操作

- 在最下一层最右边的叶子右方插入当前元素。如果最下一层已满，就新增一层。
- 但插入后可能会破坏堆的性质。

二叉堆的插入操作

- 在最下一层最右边的叶子右方插入当前元素。如果最下一层已满，就新增一层。
- 但插入后可能会破坏堆的性质。
- 于是需要**向上调整**：如果这个结点的权值小于它父亲的权值，就交换它与它父亲，重复此过程直到不满足或者到根。

二叉堆的插入操作

- 在最下一层最右边的叶子右方插入当前元素。如果最下一层已满，就新增一层。
- 但插入后可能会破坏堆的性质。
- 于是需要**向上调整**：如果这个结点的权值小于它父亲的权值，就交换它与它父亲，重复此过程直到不满足或者到根。
- 可以证明，插入之后向上调整后，没有其他结点会不满足堆性质。

二叉堆的插入操作

- 在最下一层最右边的叶子右方插入当前元素。如果最下一层已满，就新增一层。
- 但插入后可能会破坏堆的性质。
- 于是需要**向上调整**：如果这个结点的权值小于它父亲的权值，就交换它与它父亲，重复此过程直到不满足或者到根。
- 可以证明，插入之后向上调整后，没有其他结点会不满足堆性质。
- 时间复杂度 $\mathcal{O}(\log n)$ 。

```
void Insert(int v)
{
    heap[++m]=v;
    int now=m;
    while (now>1)
    {
        int nxt=now/2;
        if (heap[nxt]<=heap[now]) return;
        swap(heap[nxt],heap[now]);
        now=nxt;
    }
}
```

二叉堆的删除操作

- 朴素的二叉堆只支持删除堆顶元素，即最小值。

二叉堆的删除操作

- 朴素的二叉堆只支持删除堆顶元素，即最小值。
- 一般做法是交换根结点和最后一层最右边的元素，然后直接删除最后一个结点。

二叉堆的删除操作

- 朴素的二叉堆只支持删除堆顶元素，即最小值。
- 一般做法是交换根结点和最后一层最右边的元素，然后直接删除最后一个结点。
- 但新的根结点可能不满足堆的性质。

二叉堆的删除操作

- 朴素的二叉堆只支持删除堆顶元素，即最小值。
- 一般做法是交换根结点和最后一层最右边的元素，然后直接删除最后一个结点。
- 但新的根结点可能不满足堆的性质。
- 于是需要**向下调整**：在该结点的儿子中，找一个最大的，与该结点交换，重复此过程直到底层。

二叉堆的删除操作

- 朴素的二叉堆只支持删除堆顶元素，即最小值。
- 一般做法是交换根结点和最后一层最右边的元素，然后直接删除最后一个结点。
- 但新的根结点可能不满足堆的性质。
- 于是需要**向下调整**：在该结点的儿子中，找一个最大的，与该结点交换，重复此过程直到底层。
- 可以证明，删除并向下调整后，没有其他结点不满足堆性质。

二叉堆的删除操作

- 朴素的二叉堆只支持删除堆顶元素，即最小值。
- 一般做法是交换根结点和最后一层最右边的元素，然后直接删除最后一个结点。
- 但新的根结点可能不满足堆的性质。
- 于是需要**向下调整**：在该结点的儿子中，找一个最大的，与该结点交换，重复此过程直到底层。
- 可以证明，删除并向下调整后，没有其他结点不满足堆性质。
- 时间复杂度 $\mathcal{O}(\log n)$ 。

```
void Delete()
{
    heap[1]=heap[m--];
    int now=1;
    while (now*2<=m)
    {
        int nxt=now*2;
        if (heap[nxt]>heap[nxt+1]&&nx<m)  nxt++;
        if (heap[nxt]>=heap[now])  return;
        swap(heap[nxt],heap[now]);
        now=nxt;
    }
}
```

减小某个点的权值

- 直接修改后，向上调整一次即可。

减小某个点的权值

- 直接修改后，向上调整一次即可。
- 时间复杂度 $\mathcal{O}(\log n)$ 。

建堆

- 直接建堆时间复杂度是 $\mathcal{O}(n \log n)$ 的，这里介绍一种 $\mathcal{O}(n)$ 的方法。

建堆

- 直接建堆时间复杂度是 $\mathcal{O}(n \log n)$ 的，这里介绍一种 $\mathcal{O}(n)$ 的方法。
- 首先按顺序直接将所有元素加入一棵完全二叉树中，不需要调整。

建堆

- 直接建堆时间复杂度是 $\mathcal{O}(n \log n)$ 的，这里介绍一种 $\mathcal{O}(n)$ 的方法。
- 首先按顺序直接将所有元素加入一棵完全二叉树中，不需要调整。
- 然后考虑从叶子开始往上遍历，依次向下调整。

建堆

- 直接建堆时间复杂度是 $\mathcal{O}(n \log n)$ 的，这里介绍一种 $\mathcal{O}(n)$ 的方法。
- 首先按顺序直接将所有元素加入一棵完全二叉树中，不需要调整。
- 然后考虑从叶子开始往上遍历，依次向下调整。
- 总复杂度

$$= n \log n - \log 1 - \log 2 - \cdots - \log n \quad (1)$$

$$\leq n \log n - 0 \times 2^0 - 1 \times 2^1 - \cdots - (\log n - 1) \times \frac{n}{2} \quad (2)$$

$$= n \log n - (n - 1) - (n - 2) - (n - 4) - \cdots - (n - \frac{n}{2}) \quad (3)$$

$$= n \log n - n \log n + 1 + 2 + 4 + \cdots + \frac{n}{2} \quad (4)$$

$$= n - 1 \quad (5)$$

$$= \mathcal{O}(n) \quad (6)$$

建堆

- 直接建堆时间复杂度是 $\mathcal{O}(n \log n)$ 的，这里介绍一种 $\mathcal{O}(n)$ 的方法。
- 首先按顺序直接将所有元素加入一棵完全二叉树中，不需要调整。
- 然后考虑从叶子开始往上遍历，依次向下调整。
- 总复杂度

$$= n \log n - \log 1 - \log 2 - \cdots - \log n \quad (1)$$

$$\leq n \log n - 0 \times 2^0 - 1 \times 2^1 - \cdots - (\log n - 1) \times \frac{n}{2} \quad (2)$$

$$= n \log n - (n - 1) - (n - 2) - (n - 4) - \cdots - (n - \frac{n}{2}) \quad (3)$$

$$= n \log n - n \log n + 1 + 2 + 4 + \cdots + \frac{n}{2} \quad (4)$$

$$= n - 1 \quad (5)$$

$$= \mathcal{O}(n) \quad (6)$$

- 这东西好像只在初赛有点用。

CF1428E Carrots for Rabbits

- 新加坡动物园里有一些兔。为了喂养他们，管理员买了 n 根胡萝卜 $a_1, a_2, a_3, \dots, a_n$ 。
- 然而，兔很肥且繁殖很快。管理员现在有 k 只兔却没有足够的胡萝卜去喂它们。出于某种原因，所有胡萝卜的长度为正整数。
- 兔很难拿稳大萝卜，也很难咽下大萝卜，所以吃掉一个长度为 x 的萝卜所需要的时间是 x^2 。
- 现在请你帮助管理员切这些萝卜，以最小化兔吃萝卜的总时间。
- $1 \leq n \leq k \leq 10^5, 1 \leq a_i \leq 10^6$ 。

CF1428E Carrots for Rabbits

- 可以发现如果要把一个萝卜切开，最优策略一定是切成尽量均匀的。

CF1428E Carrots for Rabbits

- 可以发现如果要把一个萝卜切开，最优策略一定是切成尽量均匀的。
- 记 $f(x, y)$ 表示将长度为 x 的萝卜切成 y 后的最小平方和。

CF1428E Carrots for Rabbits

- 可以发现如果要把一个萝卜切开，最优策略一定是切成尽量均匀的。
- 记 $f(x, y)$ 表示将长度为 x 的萝卜切成 y 后的最小平方和。
- 容易发现 $f(x, y) - f(x, y + 1)$ 随 y 的变化是单调不增的。

CF1428E Carrots for Rabbits

- 可以发现如果要把一个萝卜切开，最优策略一定是切成尽量均匀的。
- 记 $f(x, y)$ 表示将长度为 x 的萝卜切成 y 后的最小平方和。
- 容易发现 $f(x, y) - f(x, y + 1)$ 随 y 的变化是单调不增的。
- 于是可以用一个大根堆维护对于每个萝卜，初始分段数为 1，以多分一段能减少的时间作为权值，一直取最优的统计贡献即可。

SPOJ16254 RMID2 - Running Median Again

- 维护一个序列，支持两种操作：
 - ① 向序列中插入一个元素。
 - ② 输出并删除当前序列的中位数（若序列长度为偶数，则输出较小的中位数）。
- 最终序列长度 $\leq 10^5$ ，2 操作个数 $\leq 10^5$ 。

SPOJ16254 RMID2 - Running Median Again

- 问题可以抽象成：动态维护序列第 k 大的数， k 值可能会发生变化，但 $|\Delta k|$ 之和小于等于 n 。

SPOJ16254 RMID2 - Running Median Again

- 问题可以抽象成：动态维护序列第 k 大的数， k 值可能会发生变化，但 $|\Delta k|$ 之和小于等于 n 。
- 考虑维护一个大根堆与一个小根堆，大根堆维护前 k 小的值（包含第 k 个），小根堆维护比第 k 大数大的值。

SPOJ16254 RMID2 - Running Median Again

- 问题可以抽象成：动态维护序列第 k 大的数， k 值可能会发生变化，但 $|\Delta k|$ 之和小于等于 n 。
- 考虑维护一个大根堆与一个小根堆，大根堆维护前 k 小的值（包含第 k 个），小根堆维护比第 k 大数大的值。
- 当大根堆的大小小于 k 时，不断将小根堆堆顶元素取出并插入大根堆，直到大根堆的大小等于 k ；当大根堆的大小大于 k 时，不断将大根堆堆顶元素取出并插入小根堆，直到大根堆的大小等于 k 。

SPOJ16254 RMID2 - Running Median Again

- 问题可以抽象成：动态维护序列第 k 大的数， k 值可能会发生变化，但 $|\Delta k|$ 之和小于等于 n 。
- 考虑维护一个大根堆与一个小根堆，大根堆维护前 k 小的值（包含第 k 个），小根堆维护比第 k 大数大的值。
- 当大根堆的大小小于 k 时，不断将小根堆堆顶元素取出并插入大根堆，直到大根堆的大小等于 k ；当大根堆的大小大于 k 时，不断将大根堆堆顶元素取出并插入小根堆，直到大根堆的大小等于 k 。
- 我们将这样的操作称为一次**调整**。

- 有了调整操作之后其它操作就很好做了：

SPOJ16254 RMID2 - Running Median Again

- 有了调整操作之后其它操作就很好做了：
- 插入元素：若插入的元素大于大根堆堆顶元素，则将其插入小根堆，否则将其插入大根堆，然后进行一次调整。

SPOJ16254 RMID2 - Running Median Again

- 有了调整操作之后其它操作就很好做了：
- 插入元素：若插入的元素大于大根堆堆顶元素，则将其插入小根堆，否则将其插入大根堆，然后进行一次调整。
- 查询第 k 大元素：大根堆堆顶元素即为所求。

SPOJ16254 RMID2 - Running Median Again

- 有了调整操作之后其它操作就很好做了：
- 插入元素：若插入的元素大于大根堆堆顶元素，则将其插入小根堆，否则将其插入大根堆，然后进行一次调整。
- 查询第 k 大元素：大根堆堆顶元素即为所求。
- 删除第 k 大元素：删除大根堆堆顶元素，然后进行一次调整。

SPOJ16254 RMID2 - Running Median Again

- 有了调整操作之后其它操作就很好做了：
- 插入元素：若插入的元素大于大根堆堆顶元素，则将其插入小根堆，否则将其插入大根堆，然后进行一次调整。
- 查询第 k 大元素：大根堆堆顶元素即为所求。
- 删除第 k 大元素：删除大根堆堆顶元素，然后进行一次调整。
- 修改 k 的值：直接根据新的 k 值调整。

SPOJ16254 RMID2 - Running Median Again

- 有了调整操作之后其它操作就很好做了：
- 插入元素：若插入的元素大于大根堆堆顶元素，则将其插入小根堆，否则将其插入大根堆，然后进行一次调整。
- 查询第 k 大元素：大根堆堆顶元素即为所求。
- 删除第 k 大元素：删除大根堆堆顶元素，然后进行一次调整。
- 修改 k 的值：直接根据新的 k 值调整。
- 由这两个堆组成的数据结构被称为**对顶堆**。

- 左偏树是一种可并堆，具有堆的性质，并且可以快速合并。

左偏树

- 左偏树是一种可并堆，具有堆的性质，并且可以快速合并。
- 对于一棵二叉树，我们对于每个结点维护两个值： $value$ 和 $dist$ 。

左偏树

- 左偏树是一种可并堆，具有堆的性质，并且可以快速合并。
- 对于一棵二叉树，我们对于每个结点维护两个值： $value$ 和 $dist$ 。
- $value$ 即为需要维护的权值， $dist$ 为这个结点到它子树里面最近的叶子结点的距离，叶子结点的 $dist$ 为 0。

左偏树

- 左偏树是一种可并堆，具有堆的性质，并且可以快速合并。
- 对于一棵二叉树，我们对于每个结点维护两个值： $value$ 和 $dist$ 。
- $value$ 即为需要维护的权值， $dist$ 为这个结点到它子树里面最近的叶子结点的距离，叶子结点的 $dist$ 为 0。
- 它有这么几条性质：

左偏树

- 左偏树是一种可并堆，具有堆的性质，并且可以快速合并。
- 对于一棵二叉树，我们对于每个结点维护两个值： $value$ 和 $dist$ 。
- $value$ 即为需要维护的权值， $dist$ 为这个结点到它子树里面最近的叶子结点的距离，叶子结点的 $dist$ 为 0。
- 它有这么几条性质：
 - ① 一个结点的 $value$ 一定小于等于其儿子的 $value$ (堆性质)。

左偏树

- 左偏树是一种可并堆，具有堆的性质，并且可以快速合并。
- 对于一棵二叉树，我们对于每个结点维护两个值： $value$ 和 $dist$ 。
- $value$ 即为需要维护的权值， $dist$ 为这个结点到它子树里面最近的叶子结点的距离，叶子结点的 $dist$ 为 0。
- 它有这么几条性质：
 - ① 一个结点的 $value$ 一定小于等于其儿子的 $value$ (堆性质)。
 - ② 一个结点的左儿子的 $dist$ 大于等于右儿子的 $dist$ (左偏性质)。

左偏树

- 左偏树是一种可并堆，具有堆的性质，并且可以快速合并。
- 对于一棵二叉树，我们对于每个结点维护两个值： $value$ 和 $dist$ 。
- $value$ 即为需要维护的权值， $dist$ 为这个结点到它子树里面最近的叶子结点的距离，叶子结点的 $dist$ 为 0。
- 它有这么几条性质：
 - ① 一个结点的 $value$ 一定小于等于其儿子的 $value$ (堆性质)。
 - ② 一个结点的左儿子的 $dist$ 大于等于右儿子的 $dist$ (左偏性质)。
 - ③ 一个结点的距离始终等于右儿子 +1。

左偏树

- 左偏树是一种可并堆，具有堆的性质，并且可以快速合并。
- 对于一棵二叉树，我们对于每个结点维护两个值： $value$ 和 $dist$ 。
- $value$ 即为需要维护的权值， $dist$ 为这个结点到它子树里面最近的叶子结点的距离，叶子结点的 $dist$ 为 0。
- 它有这么几条性质：
 - ① 一个结点的 $value$ 一定小于等于其儿子的 $value$ (堆性质)。
 - ② 一个结点的左儿子的 $dist$ 大于等于右儿子的 $dist$ (左偏性质)。
 - ③ 一个结点的距离始终等于右儿子 +1。
- 这样的一棵二叉树就是左偏树。

引理

结点数为 n 的左偏树，最大的 $dist$ 是 $\log n$ 级别的。

左偏树

引理

结点数为 n 的左偏树，最大的 $dist$ 是 $\log n$ 级别的。

证明.

考虑一个 $dist = i$ 的结点，他的两个儿子的 $dist$ 都至少为 $i - 1$ 。记 $size_i$ 为 $dist = i$ 的结点子树大小。也就是说 $size_i$ 至少为 $2size_{i-1} + 1$ 。由此易得 $dist$ 是 $\log n$ 级别的。□

左偏树の合并

- merge 是整个左偏树的重头戏，时间复杂度稳定在一个 $\log$ ，其主要思想就是不断把新的堆合并到新的根结点的右子树中——因为我们的右子树决定“距离”这个变量，而距离又一定保证在 $\log$ 的复杂度内，所以不断向右子树合并。

左偏树の合并

- merge 是整个左偏树的重头戏，时间复杂度稳定在一个 $\log$ ，其主要思想就是不断把新的堆合并到新的根结点的右子树中——因为我们的右子树决定“距离”这个变量，而距离又一定保证在 $\log$ 的复杂度内，所以不断向右子树合并。
- 首先我们找到要合并的两个堆的根结点，记为 x 和 y 。

左偏树の合并

- merge 是整个左偏树的重头戏，时间复杂度稳定在一个 $\log$ ，其主要思想就是不断把新的堆合并到新的根结点的右子树中——因为我们的右子树决定“距离”这个变量，而距离又一定保证在 $\log$ 的复杂度内，所以不断向右子树合并。
- 首先我们找到要合并的两个堆的根结点，记为 x 和 y 。
- 首先保证 x 的权值小于等于 y ，不满足则 $\text{swap}(x,y)$ 。把 x 作为新树的根结点，剩下的事就是合并 x 的右子树和 y 了。可以递归解决。

左偏树的合并

- merge 是整个左偏树的重头戏，时间复杂度稳定在一个 $\log$ ，其主要思想就是不断把新的堆合并到新的根结点的右子树中——因为我们的右子树决定“距离”这个变量，而距离又一定保证在 $\log$ 的复杂度内，所以不断向右子树合并。
- 首先我们找到要合并的两个堆的根结点，记为 x 和 y 。
- 首先保证 x 的权值小于等于 y ，不满足则 $\text{swap}(x,y)$ 。把 x 作为新树的根结点，剩下的事就是合并 x 的右子树和 y 了。可以递归解决。
- 合并了 x 的右子树和 y 之后，当 x 的右子树的 dist 大于 x 的左子树的 dist 时，为了维护性质二，我们要交换 x 的左右儿子。顺便维护性质三，所以直接 $\text{dist}_x = \text{dist}_{\text{rightson}(x)} + 1$ 。

左偏树的合并

- merge 是整个左偏树的重头戏，时间复杂度稳定在一个 $\log$ ，其主要思想就是不断把新的堆合并到新的根结点的右子树中——因为我们的右子树决定“距离”这个变量，而距离又一定保证在 $\log$ 的复杂度内，所以不断向右子树合并。
- 首先我们找到要合并的两个堆的根结点，记为 x 和 y 。
- 首先保证 x 的权值小于等于 y ，不满足则 $\text{swap}(x,y)$ 。把 x 作为新树的根结点，剩下的事就是合并 x 的右子树和 y 了。可以递归解决。
- 合并了 x 的右子树和 y 之后，当 x 的右子树的 dist 大于 x 的左子树的 dist 时，为了维护性质二，我们要交换 x 的左右儿子。顺便维护性质三，所以直接 $\text{dist}_x = \text{dist}_{\text{rightson}(x)} + 1$ 。
- 递归次数 = 两个堆根结点的 dist 之和，因此是 $\mathcal{O}(\log n)$ 的。

左偏树的合并

- merge 是整个左偏树的重头戏，时间复杂度稳定在一个 $\log$ ，其主要思想就是不断把新的堆合并到新的根结点的右子树中——因为我们的右子树决定“距离”这个变量，而距离又一定保证在 $\log$ 的复杂度内，所以不断向右子树合并。
- 首先我们找到要合并的两个堆的根结点，记为 x 和 y 。
- 首先保证 x 的权值小于等于 y ，不满足则 $\text{swap}(x,y)$ 。把 x 作为新树的根结点，剩下的事就是合并 x 的右子树和 y 了。可以递归解决。
- 合并了 x 的右子树和 y 之后，当 x 的右子树的 dist 大于 x 的左子树的 dist 时，为了维护性质二，我们要交换 x 的左右儿子。顺便维护性质三，所以直接 $\text{dist}_x = \text{dist}_{\text{rightson}(x)} + 1$ 。
- 递归次数 = 两个堆根结点的 dist 之和，因此是 $\mathcal{O}(\log n)$ 的。
- Tips: 找根结点的时候要加路径压缩，因为左偏树的树高可能会退化到 $\mathcal{O}(n)$ 。

```
int merge(int x,int y)
{
    if (x==y) return x;
    if (!x||!y) return x|y;
    if (t[x].val>t[y].val||(t[x].val==t[y].val&& x>y)
        ) swap(x,y);
    t[x].rs=merge(t[x].rs,y);
    if (t[t[x].ls].dis<t[t[x].rs].dis) swap(t[x].ls,
        t[x].rs);
    t[t[x].ls].fa=t[t[x].rs].fa=t[x].fa=x;
    t[x].dis=t[t[x].rs].dis+1;
    return x;
}
```

左偏树の删根

- 因为左偏树支持合并，因此删除根结点操作非常好做。

左偏树の删根

- 因为左偏树支持合并，因此删除根结点操作非常好做。
- 只需要直接删除根结点，然后合并左右儿子即可

左偏树の删根

- 因为左偏树支持合并，因此删除根结点操作非常好做。
- 只需要直接删除根结点，然后合并左右儿子即可
- **Tips:** 因为有路径压缩的存在，所以需要将根结点的父亲指向新堆的根，否则会出大问题。

删除任意结点

- 记要删除的结点为 x ，按照删根的方法处理 x 的子树，然后自底向上更新 $dist$ 、不满足左偏性质时交换左右儿子，当 $dist$ 无需更新时结束递归。

删除任意结点

- 记要删除的结点为 x ，按照删根的方法处理 x 的子树，然后自底向上更新 $dist$ 、不满足左偏性质时交换左右儿子，当 $dist$ 无需更新时结束递归。
- 复杂度也是单 $\log$ 的

删除任意结点

- 记要删除的结点为 x ，按照删根的方法处理 x 的子树，然后自底向上更新 $dist$ 、不满足左偏性质时交换左右儿子，当 $dist$ 无需更新时结束递归。
- 复杂度也是单 $\log$ 的
- 证明可以考虑如果一个结点 x 需要向上继续更新的话，它父亲的 $dist$ 一定等于当前结点的 $dist + 1$ ，而 $dist$ 是 $\mathcal{O}(\log n)$ 的

整个堆加上/减去一个值、乘上一个正数

- 其实对于可以打标记且不改变相对大小的操作都可以做到。

整个堆加上/减去一个值、乘上一个正数

- 其实对于可以打标记且不改变相对大小的操作都可以做到。
- 在根打上标记，删除根/合并堆 (访问儿子) 时下传标记即可。

目录

- 1 栈
- 2 队列
- 3 链表

- 4 桶
- 5 并查集
- 6 堆
- 7 哈希表

- 哈希表是基于哈希函数建立的一种查找表。类似于一个不连续的数组。

- 哈希表是基于哈希函数建立的一种查找表。类似于一个不连续的数组。
- 可以直接使用 C++11 中的 `unordered_map`。

哈希表

- 哈希表是基于哈希函数建立的一种查找表。类似于一个不连续的数组。
- 可以直接使用 C++11 中的 `unordered_map`。
- 但 NOIP 没开过 C++11。

哈希表

- 哈希表是基于哈希函数建立的一种查找表。类似于一个不连续的数组。
- 可以直接使用 C++11 中的 `unordered_map`。
- 但 NOIP 没开过 C++11。
- 所以你需要学会手写哈希表。

- 一般哈希表的哈希函数都是 $hash(key) = (a \times key + b) \bmod p$

哈希表

- 一般哈希表的哈希函数都是 $hash(key) = (a \times key + b) \bmod p$
- 其中 a, b 取任意正整数, p 根据内存取合适的质数。

哈希表

- 一般哈希表的哈希函数都是 $hash(key) = (a \times key + b) \bmod p$
- 其中 a, b 取任意正整数, p 根据内存取合适的质数。
- 然后直接根据哈希值丢到一个数组中即可。

哈希表

- 一般哈希表的哈希函数都是 $hash(key) = (a \times key + b) \bmod p$
- 其中 a, b 取任意正整数, p 根据内存取合适的质数。
- 然后直接根据哈希值丢到一个数组中即可。
- 但这样显然会产生冲突。

哈希表

- 一般哈希表的哈希函数都是 $hash(key) = (a \times key + b) \bmod p$
- 其中 a, b 取任意正整数, p 根据内存取合适的质数。
- 然后直接根据哈希值丢到一个数组中即可。
- 但这样显然会产生冲突。
- 在冲突时一直令 哈希值 = (哈希值 + 5) $\bmod p$, 直到不冲突即可。

哈希表

- 一般哈希表的哈希函数都是 $hash(key) = (a \times key + b) \bmod p$
- 其中 a, b 取任意正整数, p 根据内存取合适的质数。
- 然后直接根据哈希值丢到一个数组中即可。
- 但这样显然会产生冲突。
- 在冲突时一直令 哈希值 = (哈希值 + 5) $\bmod p$, 直到不冲突即可。
- 当然你不一定要取 5, 任何小于 p 的正整数都可以。


```

struct hash
{
    ull key[p];
    int val[p];
    hash(){memset(key,-1,sizeof(key));}
    void insert(ull x)
    {
        ull k=x%p;
        while (~key[k]&&key[k]!=x) add(k,5);
        key[k]=x; val[k]++;
    }
    int query(ull x)
    {
        ull k=x%p;
        while (~key[k]&&key[k]!=x) add(k,5);
        return val[k];
    }
}h;

```

某场 sxyz WC 模拟赛 T1 棋盘旅行

- 有一个 8×8 的棋盘，每个格子上有个小写字母。
- 给出一个长度为 k 的目标串，记为 $s_1, s_2, \dots, s_k$ 。
- 求有多少条国王 (国际象棋中) 的旅行路线满足路径上经过的字符组成的字符串恰好为给出的目标串，且不重复经过同一个格子。
- $k \leq 11$ 。

Solution

- 考虑 meet-in-middle。

Solution

- 考虑 meet-in-middle。
- 搜出所有前半条合法的路径，将格子经过/不经过的状态压入一个 ull 中，存到这半条路径的终点的位置。

Solution

- 考虑 meet-in-middle。
- 搜出所有前半条合法的路径，将格子经过/不经过的状态压入一个 ull 中，存到这半条路径的终点的位置。
- 同样方法处理出后半条路径的状态，现在就要是对于每个后半条的路径，求出有多少条存储在路径交点处的前半条路径，与之不相交。

Solution

- 考虑 meet-in-middle。
- 搜出所有前半条合法的路径，将格子经过/不经过的状态压入一个 ull 中，存到这半条路径的终点的位置。
- 同样方法处理出后半条路径的状态，现在就要是对于每个后半条的路径，求出有多少条存储在路径交点处的前半条路径，与之不相交。
- 考虑容斥。记当前后半条路径经过的点集为 S ，显然答案为
$$\sum_{T \in S} (-1)^{|T|} \text{包含 } T \text{ 的合法前半条路径个数。}$$

Solution

- 考虑 meet-in-middle。
- 搜出所有前半条合法的路径，将格子经过/不经过的状态压入一个 ull 中，存到这半条路径的终点的位置。
- 同样方法处理出后半条路径的状态，现在就要是对于每个后半条的路径，求出有多少条存储在路径交点处的前半条路径，与之不相交。
- 考虑容斥。记当前后半条路径经过的点集为 S ，显然答案为 $\sum_{T \in S} (-1)^{|T|}$ 包含 T 的合法前半条路径个数。
- T 可以直接枚举子集，那么包含 T 的合法前半条路径个数怎么算呢？

- 考虑 meet-in-middle。
- 搜出所有前半条合法的路径，将格子经过/不经过的状态压入一个 ull 中，存到这半条路径的终点的位置。
- 同样方法处理出后半条路径的状态，现在就要是对于每个后半条的路径，求出有多少条存储在路径交点处的前半条路径，与之不相交。
- 考虑容斥。记当前后半条路径经过的点集为 S ，显然答案为 $\sum_{T \in S} (-1)^{|T|}$ 包含 T 的合法前半条路径个数。
- T 可以直接枚举子集，那么包含 T 的合法前半条路径个数怎么算呢？
- 可以在将前半条路径处理出来的时候也枚举子集，将哈希表中所有子集的位置 +1，容斥的时候直接查表即可。



GL&HF
Thanks for Listening